Labolatorium 1

March 12, 2026

Jakub Orchowski, s223281

1 CEL ĆWICZENIA

Modele kolorów i ich przekształcanie

[51]:
```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
%matplotlib inline
```

2 Zadania

[52]:
```
Im = np.array(Image.open('obraz.bmp'))
```

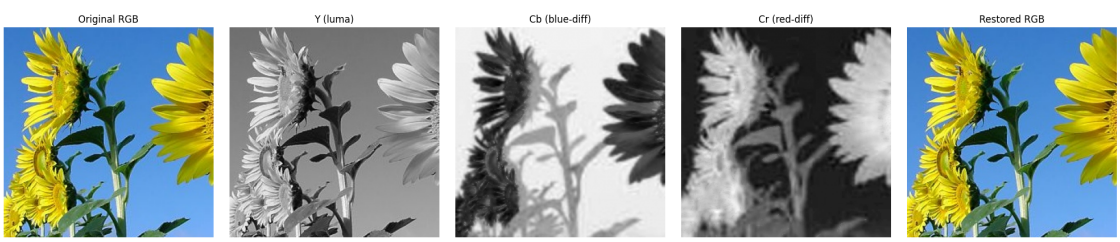
2.1 Zadanie 1.

Napisz funkcję do przekształcania obrazów w formacie RGB w inny format zdefiniowany przez macierz przekształcenia H o wymiarach 3x3

2.1.1 Format YCbCr

[53]:
```
from numpy.typing import NDArray

YCBCR_VALUES = np.array([0.299, 0.587, 0.114])

def YCbCrMatrix(values: NDArray) -> NDArray:
    [kr, kg, kb] = values
    return np.array([
        [ kr,                   kg,                   kb                  ],
        [-0.5 * kr / (1 - kb), -0.5 * kg / (1 - kb),  0.5                 ],
        [ 0.5,                 -0.5 * kg / (1 - kr), -0.5 * kb / (1 - kr) ]
    ])

def RGBtoYCbCr(image: NDArray) -> NDArray:
    return np.dot(image.astype(np.float64), YCbCrMatrix(YCBCR_VALUES).T)

def YCbCrtoRGB(ycbcr: NDArray) -> NDArray:
```

```
    H_inv = np.linalg.inv(YCbCrMatrix(YCBCR_VALUES))
    return np.clip(np.dot(ycbcr, H_inv.T), 0, 255).astype(np.uint8)

def normalize(channel: NDArray) -> NDArray:
    lo, hi = channel.min(), channel.max()
    return (channel - lo) / (hi - lo) if hi > lo else np.zeros_like(channel)

YCbCr = RGBtoYCbCr(Im)

fig, axes = plt.subplots(1, 5, figsize=(20, 4))
axes[0].imshow(Im)
axes[0].set_title('Original RGB')
axes[0].axis('off')

for i, name in enumerate(['Y (luma)', 'Cb (blue-diff)', 'Cr (red-diff)']):
    axes[i + 1].imshow(normalize(YCbCr[..., i]), cmap='gray')
    axes[i + 1].set_title(name)
    axes[i + 1].axis('off')

Im_restored = YCbCrtoRGB(YCbCr)
axes[4].imshow(Im_restored)
axes[4].set_title('Restored RGB')
axes[4].axis('off')

plt.tight_layout()
plt.show()

max_err = np.max(np.abs(Im.astype(int) - Im_restored.astype(int)))
print(f"Maksymalny blad piksela po odwroceniu YCbCr: {max_err}")
```




```
Maksymalny blad piksela po odwroceniu YCbCr: 1
```

2.1.2 Format YUV (wariant analogowy BT.601)

[54]:
```
YUV_MATRIX = np.array([
    [ 0.29900,  0.58700,  0.11400],
    [-0.14713, -0.28886,  0.43600],
    [ 0.61500, -0.51499, -0.10001],
```

```
])

def RGBtoYUV(image: np.ndarray) -> np.ndarray:
    return np.dot(image.astype(np.float64), YUV_MATRIX.T)

def YUVtoRGB(yuv: np.ndarray) -> np.ndarray:
    H_inv = np.linalg.inv(YUV_MATRIX)
    return np.clip(np.dot(yuv, H_inv.T), 0, 255).astype(np.uint8)

YUV = RGBtoYUV(Im)

fig, axes = plt.subplots(1, 5, figsize=(20, 4))
axes[0].imshow(Im)
axes[0].set_title('Original RGB')
axes[0].axis('off')

for i, name in enumerate(['Y (luma)', 'U (blue-proj)', 'V (red-proj)']):
    axes[i + 1].imshow(normalize(YUV[..., i]), cmap='gray')
    axes[i + 1].set_title(name)
    axes[i + 1].axis('off')

Im_restored_yuv = YUVtoRGB(YUV)
axes[4].imshow(Im_restored_yuv)
axes[4].set_title('Restored RGB')
axes[4].axis('off')

plt.tight_layout()
plt.show()

max_err = np.max(np.abs(Im.astype(int) - Im_restored_yuv.astype(int)))
print(f"Maksymalny blad piksela po odwroceniu YUV: {max_err}")
```

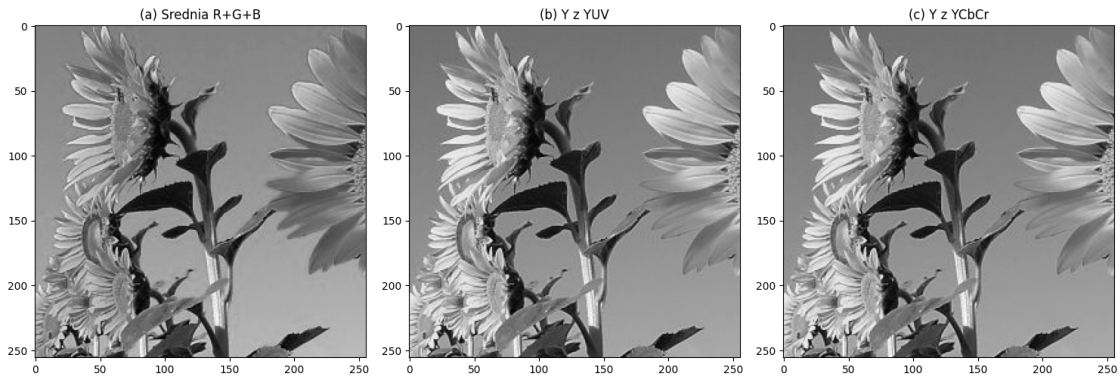
```
Maksymalny blad piksela po odwroceniu YUV: 1
```

2.2 Zadanie 2.

Zamień obraz kolorowy RGB w obraz monochromatyczny trzema metodami: - **(a)** średnia arytmetyczna kanałów R, G, B

- **(b)** kanał Y z formatu YUV (Zadanie 1a)
- **(c)** kanał Y z formatu YCbCr (Zadanie 1b)

[58]:
```
gray_avg   = Im.astype(np.float64).mean(axis=2)
gray_yuv   = YUV[..., 0]
gray_ycbcr = YCbCr[..., 0]

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for ax, img, title in zip(axes,
                          [gray_avg, gray_yuv, gray_ycbcr],
                          ['(a) Srednia R+G+B', '(b) Y z YUV', '(c) Y z␣
 ↪YCbCr']):
    ax.imshow(normalize(img), cmap='gray')
    ax.set_title(title)

plt.tight_layout()
plt.show()
```



2.2.1 Wnioski

Metoda (a) traktuje R, G i B rownorzedie (wagi 1/3 kazdy).

Metody (b) i (c) uzywaja kombinacji Y = 0.299R + 0.587G + 0.114B, ktora modeluje luminancje zgodnie z wraxliwoscia ludzkiego oka.

W efekcie (b) i (c) daja praktycznie identyczny obraz, natomiast (a) rozni sie jasnoscia poszczegolnych barw.

2.3 Zadanie 3.

Dane są trzy obrazy monochromatyczne `A_RGB.bmp`, `A_YCbCr.bmp`, `A_YUV.bmp` uzyskane z tego samego obrazu RGB metodami z Zadania 2. Na ich podstawie odtworzyć oryginalny obraz RGB.

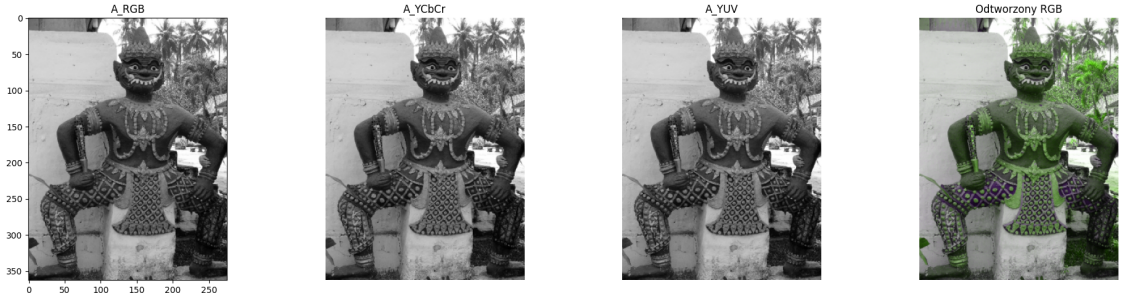
Każdy z pikselowych wartości spełnia:

```
a_avg    = (R + G + B) / 3
```

```
a_YCbCr  = 0.299·R + 0.587·G + 0.114·B
a_YUV    = 0.299·R + 0.587·G + 0.114·B
```

Ponieważ równania 2 i 3 są identyczne, układ jest nieoznaczony – stosujemy pseudoobrót (lstsq).

[ ]:
```
from PIL import Image as PILImage

A_rgb   = np.array(PILImage.open('A_RGB.bmp').convert('L')).astype(np.float64)
A_ycbcr = np.array(PILImage.open('A_YCbCr.bmp').convert('L')).astype(np.float64)
A_yuv   = np.array(PILImage.open('A_YUV.bmp').convert('L')).astype(np.float64)

M = np.array([
    [1/3,   1/3,   1/3  ],
    [0.299, 0.587, 0.114],
    [0.299, 0.587, 0.114],
])

h, w = A_rgb.shape
b_stack = np.stack([A_rgb, A_ycbcr, A_yuv], axis=-1).reshape(-1, 3)

x, _, _, _ = np.linalg.lstsq(M, b_stack.T, rcond=None)
Im_recovered = np.clip(x.T.reshape(h, w, 3), 0, 255).astype(np.uint8)

fig, axes = plt.subplots(1, 4, figsize=(20, 5))
axes[0].imshow(A_rgb,   cmap='gray')
axes[0].set_title('A_RGB')
axes[0].axis('off')
axes[1].imshow(A_ycbcr, cmap='gray')
axes[1].set_title('A_YCbCr')
axes[1].axis('off')
axes[2].imshow(A_yuv,   cmap='gray')
axes[2].set_title('A_YUV')
axes[2].axis('off')
axes[3].imshow(Im_recovered)
axes[3].set_title('Odtworzony RGB')
axes[3].axis('off')
plt.tight_layout()
plt.show()
```



2.3.1 Wnioski

Rowniania 2 i 3 sa identyczne (ten sam Y), wiec uklad jest nieoznaczony.

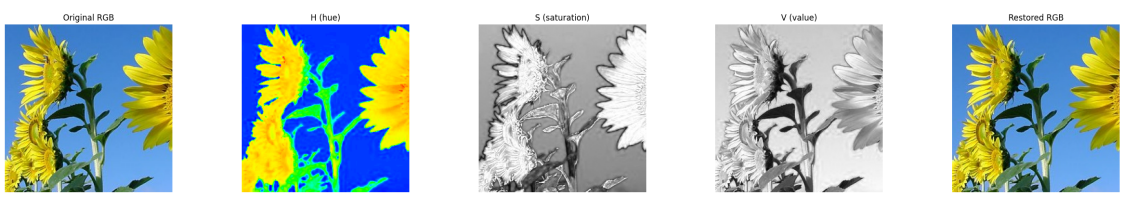
Rozwiazanie lstsq daje przyblizenie o minimalnej normie.

2.4 Zadanie 4.

Format HSV: konwersja RGB → HSV → RGB oraz obliczenie współrzędnych HSV wybranych kolorów.

[57]:
```
import matplotlib.colors as mcolors
import pandas as pd

Im_float        = Im.astype(np.float64) / 255.0
Im_hsv          = mcolors.rgb_to_hsv(Im_float)
Im_rgb_restored = mcolors.hsv_to_rgb(Im_hsv)

fig, axes = plt.subplots(1, 5, figsize=(25, 4))
axes[0].imshow(Im)
axes[0].set_title('Original RGB')
axes[0].axis('off')
axes[1].imshow(Im_hsv[..., 0], cmap='hsv')
axes[1].set_title('H (hue)')
axes[1].axis('off')
axes[2].imshow(Im_hsv[..., 1], cmap='gray')
axes[2].set_title('S (saturation)')
axes[2].axis('off')
axes[3].imshow(Im_hsv[..., 2], cmap='gray')
axes[3].set_title('V (value)')
axes[3].axis('off')
axes[4].imshow(Im_rgb_restored)
axes[4].set_title('Restored RGB')
axes[4].axis('off')
plt.tight_layout()
plt.show()

max_err = np.max(np.abs(Im_float - Im_rgb_restored))
print(f"Maksymalny blad piksela po round-trip HSV: {max_err:.2e}")

colors_rgb = np.array([
    [1, 0, 0],
    [0, 1, 0],
    [0, 0, 1],
    [0, 1, 1],
    [1, 0, 1],
```

```
    [1, 1, 0],
], dtype=np.float64)

color_names = ['Czerwony', 'Zielony', 'Niebieski', 'Siny', 'Karmazynowy',␣
 ↪'Zolty']

colors_hsv      = mcolors.rgb_to_hsv(colors_rgb.reshape(1, -1, 3)).reshape(-1,␣
 ↪3)
colors_dark_hsv = mcolors.rgb_to_hsv((colors_rgb * 0.5).reshape(1, -1, 3)).
 ↪reshape(-1, 3)

df = pd.DataFrame({
    'Kolor':   color_names,
    'R': colors_rgb[:, 0],          'G': colors_rgb[:, 1],          'B':␣
 ↪colors_rgb[:, 2],
    'H': colors_hsv[:, 0].round(4), 'S': colors_hsv[:, 1].round(4), 'V':␣
 ↪colors_hsv[:, 2].round(4),
    'H(x0.5)': colors_dark_hsv[:, 0].round(4),
    'S(x0.5)': colors_dark_hsv[:, 1].round(4),
    'V(x0.5)': colors_dark_hsv[:, 2].round(4),
})
print(df.to_string(index=False))
```




```
Maksymalny blad piksela po round-trip HSV: 4.44e-16
      Kolor   R   G   B      H   S   V  H(x0.5)  S(x0.5)  V(x0.5)
   Czerwony 1.0 0.0 0.0 0.0000 1.0 1.0   0.0000      1.0      0.5
    Zielony 0.0 1.0 0.0 0.3333 1.0 1.0   0.3333      1.0      0.5
  Niebieski 0.0 0.0 1.0 0.6667 1.0 1.0   0.6667      1.0      0.5
       Siny 0.0 1.0 1.0 0.5000 1.0 1.0   0.5000      1.0      0.5
Karmazynowy 1.0 0.0 1.0 0.8333 1.0 1.0   0.8333      1.0      0.5
      Zolty 1.0 1.0 0.0 0.1667 1.0 1.0   0.1667      1.0      0.5
```

2.4.1 Wnioski

Przyciemnienie (x0.5) zmniejsza V o polowe, H i S pozostaja bez zmian